

Skill 3:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <termios.h>
#include <unistd.h>

#define INITIAL_BUFFER_SIZE 16
#define HISTORY_SIZE 10

char *history[HISTORY_SIZE];
int history_count = 0;

void add_history(const char *command) {
    if (strlen(command) == 0)
        return;

    if (history_count < HISTORY_SIZE) {
        history[history_count] = strdup(command);
        history_count++;
    } else {
        free(history[0]);

        for (int i = 1; i < HISTORY_SIZE; i++) {
            history[i - 1] = history[i];
        }

        history[HISTORY_SIZE - 1] = strdup(command);
    }
}

void clear_history() {
    for (int i = 0; i < history_count; i++) {
        free(history[i]);
    }
}

void print_history() {
```

```

    for (int i = 0; i < history_count; i++) {
        printf("%d %s\n", i + 1, history[i]);
    }
}

int main() {
    struct termios old_terminal, new_terminal;

    char *buffer = malloc(INITIAL_BUFFER_SIZE);
    int buffer_size = INITIAL_BUFFER_SIZE;
    int length = 0;

    int history_position = history_count;

    if (buffer == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    tcgetattr(STDIN_FILENO, &old_terminal);
    new_terminal = old_terminal;

    new_terminal.c_lflag &= ~(ICANON | ECHO);

    tcsetattr(STDIN_FILENO, TCSANOW, &new_terminal);

    printf("Simple Shell History Program\n");
    printf("Type commands and press Enter.\n");
    printf("Use UP/DOWN arrows for history.\n");
    printf("Type 'history' to display history.\n");
    printf("Type 'exit' to quit.\n\n");

    while (1) {
        printf("\r> ");
        fflush(stdout);

```

```

buffer[0] = '\0';
history_position = history_count;

while (1) {

    char c;

    if (read(STDIN_FILENO, &c, 1) <= 0)
        break;

    // Enter key
    if (c == '\n' || c == '\r') {
        buffer[length] = '\0';
        printf("\n");
        break;
    }

    // Backspace
    if (c == 127 || c == 8) {
        if (length > 0) {
            length--;
            buffer[length] = '\0';

            printf("\b \b");
            fflush(stdout);
        }

        continue;
    }

    // Escape sequence
    if (c == 27) {
        char seq[2];

        if (read(STDIN_FILENO, &seq[0], 1) <= 0)
            continue;

```

```

        buffer = temp;
    }

    buffer[length++] = c;
    buffer[length] = '\0';

    putchar(c);
    fflush(stdout);
}

if (strcmp(buffer, "exit") == 0)
    break;

if (strcmp(buffer, "history") == 0) {

    add_history(buffer);
    print_history();

} else if (strlen(buffer) > 0) {

    add_history(buffer);

    printf("Command received: %s\n",
           buffer);

}

}

tcsetattr(STDIN_FILENO, TCSANOW, &old_terminal);

free(buffer);
clear_history();

printf("\nProgram terminated.\n");

return 0;
}

```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    char *command;
    struct Node *next;
} Node;

Node *create_node(const char *command) {

    Node *new_node = malloc(sizeof(Node));

    if (new_node == NULL) {
        return NULL;
    }

    new_node->command = malloc(strlen(command) + 1);

    if (new_node->command == NULL) {
        free(new_node);
        return NULL;
    }

    strcpy(new_node->command, command);

    new_node->next = NULL;

    return new_node;
}

void add_node(Node **head, const char *command) {

    Node *new_node = create_node(command);

    if (new_node == NULL) {
        printf("Memory allocation failed.\n");
    }
}

```

```
}

void free_list(Node *head) {
    Node *temp;
    while (head != NULL) {
        temp = head;
        head = head->next;
        free(temp->command);
        free(temp);
    }
}

int main() {
    Node *head = NULL;

    add_node(&head, "ls");
    add_node(&head, "pwd");
    add_node(&head, "gcc program.c");
    add_node(&head, "echo Hello");

    printf("Commands stored in linked list:\n\n");

    print_list(head);

    free_list(head);

    printf("\nAll allocated memory released successfully.\n");

    return 0;
}
```

```

bhanu@DESKTOP-DE32HQ8:~/shell_history$ valgrind --leak-check=full ./memory
==3395== Memcheck, a memory error detector
==3395== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==3395== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==3395== Command: ./memory
==3395==
Commands stored in linked list:

1. ls
2. pwd
3. gcc program.c
4. echo Hello

All allocated memory released successfully.
==3395==
==3395== HEAP SUMMARY:
==3395==      in use at exit: 0 bytes in 0 blocks
==3395==    total heap usage: 9 allocs, 9 frees, 1,120 bytes allocated
==3395==
==3395== All heap blocks were freed -- no leaks are possible
==3395==

```

```

bhanu@DESKTOP-DE32HQ8:~$ cd ~
mkdir shell_history
cd shell_history
bhanu@DESKTOP-DE32HQ8:~/shell_history$ pwd
/home/bhanu/shell_history
bhanu@DESKTOP-DE32HQ8:~/shell_history$ nano history.c
bhanu@DESKTOP-DE32HQ8:~/shell_history$ gcc -Wall -Wextra history.c -o history
bhanu@DESKTOP-DE32HQ8:~/shell_history$ ./history
Simple Shell History Program
Type commands and press Enter.
Use UP/DOWN arrows for history.
Type 'history' to display history.
Type 'exit' to quit.

> ls
Command received: ls
> pwd
Command received: pwd
> hello
Command received: hello
> history

Command History:
1  ls
2  pwd
3  hello
4  history
> pwd
bhanu@DESKTOP-DE32HQ8:~/shell_history$ nano memory.c
bhanu@DESKTOP-DE32HQ8:~/shell_history$
bhanu@DESKTOP-DE32HQ8:~/shell_history$ nano memory.c
bhanu@DESKTOP-DE32HQ8:~/shell_history$ gcc -Wall -Wextra memory.c -o memory
bhanu@DESKTOP-DE32HQ8:~/shell_history$ ./memory
Commands stored in linked list:

1. ls
2. pwd
3. gcc program.c
4. echo Hello

```

